@aman_views

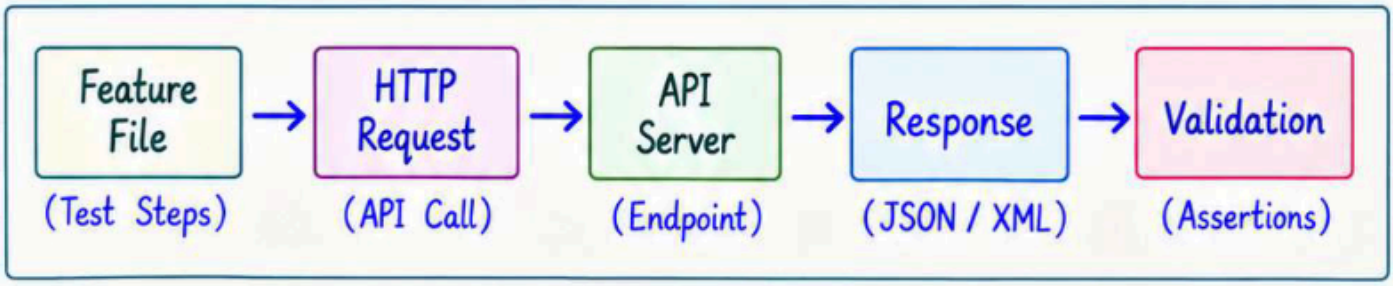
KARATE AUTOMATION NOTES

1. What is Karate ?

- Karate is an open-source test automation framework, mainly for API testing.
- Uses a readable DSL (Domain Specific Language) in Given-When-Then style.
- Supports API testing, assertions, mocks, data-driven tests and parallel execution.

Why use Karate ?

- Less Java code.
- Fast API automation.
- Easy JSON assertions.



2. Karate vs REST Assured

Aspect	Karate	REST Assured
Style	BDD-style DSL	Java library
Coding Effort	Low	More Java code
Test Format	.feature file	Java test class
Assertions	Built-in match syntax	Java assertions
Best For	Fast readable API automation	Java-heavy custom frameworks

Quick takeaway :

- Karate is great for readable API automation. REST Assured is flexible but needs more Java coding.

@aman_views

KARATE AUTOMATION NOTES

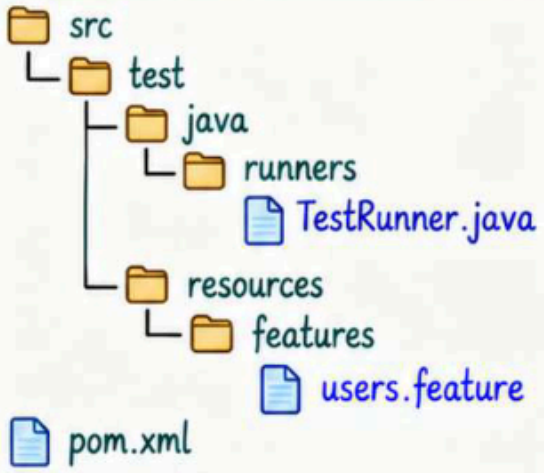
3. Karate Project Setup

- Karate is commonly set up in a Maven project.
- Add the Karate JUnit 5 dependency.
- Place feature files under test resources and runners under test java.

Maven Dependency (pom.xml)

```
<dependency>
   <groupId>com.intuit.karate</groupId>
   <artifactId>karate-junit5 </artifactId>
   <version>1.4.1</version>
   <scope>test</scope>
</dependency>
```

Project Structure



4. Feature File Structure

- A .feature file contains Feature, Background, Scenario and steps.
- Feature defines the test suite, Background has common setup, and Scenario contains the test steps.

Example feature file (users.feature)

```
Feature: User API Tests
   Background:
      * url baseUrl
   Scenario: Get user
      Given path 'users', 1
      When method get
      Then status 200
```

Key Elements

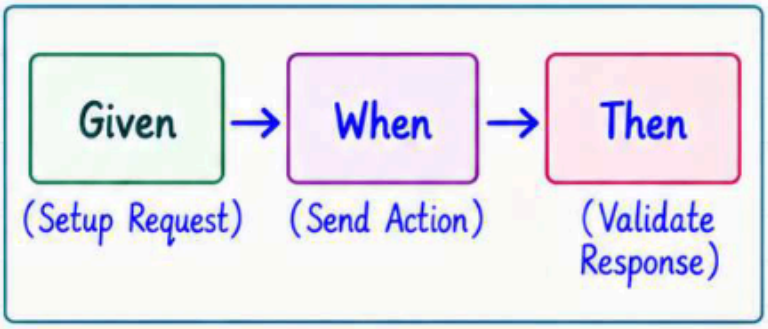
Feature	→	= test suite (group of scenarios)
Background	→	= common setup (runs before scenarios)
Scenario	→	= test case (contains test steps)

@aman_views

KARATE AUTOMATION NOTES

5. Given / When / Then

- Given prepares the request data (URL, headers, payload, etc).
- When sends the action / request to the server.
- Then validates the response (status, body, headers, etc).




```
Given url baseUrl                # set the base URL
And path 'users', 1              # set path with parameter
When method get                  # send GET request
Then status 200                  # validate response status
```

6. HTTP Methods

Method	Use
GET	Read data (retrieve resources)
POST	Create data (add new resource)
PUT	Replace / update existing data (full update)
PATCH	Partial update of existing data
DELETE	Remove data (delete resource)

- Karate supports all common HTTP methods.
- The keyword used is method (e.g. When method get).
- The API behavior depends on the endpoint and the business logic.

Example :

```
When method post
# sends a POST request
```

@aman_views

KARATE AUTOMATION NOTES

7. Headers & Query Parameters

- Headers send metadata like authorization, content type, etc.
- Query parameters pass optional values in the URL.

Example :

```
Given url baseUrl
And path 'users'
And header Authorization = 'Bearer ' + token
And param page = 2
When method get
Then status 200
```

Examples :

Header examples :

- Authorization: Bearer <token>
- Content-Type: application/json

Query parameter example :

- ?page=2

---

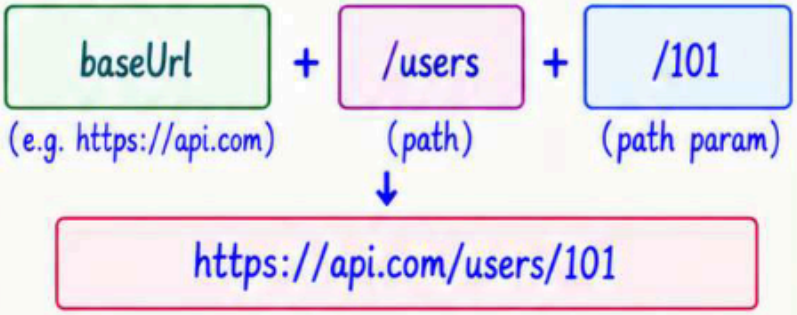
8. Path Parameters

- Path parameter values become part of the URL.
- Karate builds paths using the path keyword.

Example :

```
* def userId = 101
Given url baseUrl
And path 'users', userId
When method get
Then status 200
```

URL breakdown :



@aman_views

KARATE AUTOMATION NOTES

9. Request Body & JSON

- Request payload is sent mainly in POST / PUT / PATCH calls.
- Karate works naturally with JSON.

```
# Create a user with JSON request body
* def payload = {
    name: 'Aman',
    job: 'Tester'
  }
Given url baseUrl
And path 'users'
And request payload
When method post
Then status 201
```

Note :

- request sets the body of the HTTP call.

10. Response Validation

- Validate status code and important response fields.
- Response can be checked using match.

```
# Validate response from API
Then status 200
And match response.name == 'Aman'
And match response.id == "#string"
```



6.

KARATE AUTOMATION NOTES

11. Match Keyword

- match is the main assertion keyword in Karate.
- It can compare full objects, fields, and partial content.

```
# 1. Match full object
* match response == expectedResponse

# 2. Match a specific field
* match response.name == 'Aman'

# 3. Match partial content
* match response contains { job: 'Tester' }
```

Exact vs Contains

- match actual == expected
  → Checks for exact match.
- match actual contains { ... }
  → Checks if actual contains the given fields/values.

12. Schema Validation

- Karate fuzzy matchers validate the structure and types of a response.
- Useful when exact values are dynamic (e.g. IDs, timestamps, names).

```
# Define a schema with fuzzy matchers
* def userSchema = {
      id: '#number',
      name: '#string',
      active: '#boolean'
  }

# Validate response against schema
* match response == userSchema
```

Common Fuzzy Markers

Marker	Validates
#string	A string value
#number	A numeric value
#boolean	A boolean value (true / false)
#array	An array (list)

KARATE AUTOMATION NOTES

13. Variables & Expressions

- Use def to create variables in Karate.
- Variables make tests reusable and dynamic.

```
* def userId = 10
* def userName = 'Aman'

Given path 'users', userId
And request { name: '#(userName)' }
```

Note: #(variable) embeds a value in JSON.

14. Reading External Files

- read() loads payloads or test data from files.
- Helps keep tests clean and easy to maintain.

```
* def userPayload = read('payloads/user.json')
* def csvData = read('data/users.csv')
```



@aman_views

KARATE AUTOMATION NOTES

15. karate-config.js

- This file stores common configuration values for the tests.
- It runs before tests and can return variables that can be used in feature files.

```
function fn() {
  var config = {
      baseUrl: "https://reqres.in/api"
  };
  return config;
}
```

Note :

- Feature files can use baseUrl directly (e.g. ${baseUrl}).

16. Environment Configuration

- karate.env helps switch between dev, qa and prod environments.
- Use separate base URLs or settings per environment.

Environment	Base URL
dev	https://dev.api.com
qa	https://qa.api.com
prod	https://api.com

Run with environment:

```
mvn test -Dkarate.env=qa
```

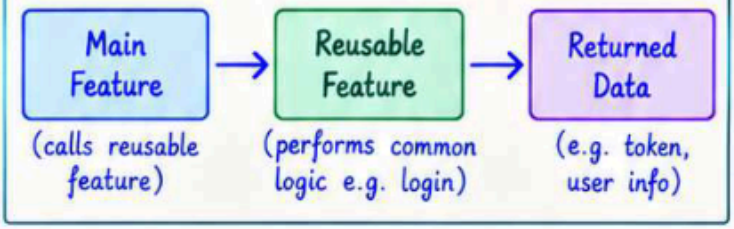
karate.env (dev / qa / prod) → Select Config (Base URL / Settings) → Run Tests (With selected config)

KARATE AUTOMATION NOTES

17. Reusable Feature Files

- Common logic can be reused across multiple tests.
- Useful for login, common headers, test-data creation, etc.
- Reduces duplication and makes tests easier to maintain.

Flow: Reusing a Feature



auth.feature (Reusable Feature)

```
Feature: Login and get auth token

Scenario: Login
  Given url "https://reqres.in/api"
  And request {
        "email": "test@reqres.in",
        "password": "password123"
  }
  When method post
  Then status 200
  * def token = response.token
  * def result = { token: '#(token)' }
```

users.feature (Using Reusable Feature)

```
Feature: Users API Tests

Background:
  * def auth = call read('auth.feature')
  * def token = auth.token

Scenario: Get all users
  Given url 'https://reqres.in/api/users'
  And header Authorization = 'Bearer ' + token
  When method get
  Then status 200
  And print response
```

18. call vs callonce

Aspect	call	callonce
Execution	Runs every time it is called (in each scenario).	Runs only once per feature file (then caches the result).
Best for	Fresh / dynamic data (e.g. create new test data).	Common one-time setup (e.g. login, get token, load config).
Example use	Create new user, generate unique data, get latest info.	Login once, get auth token, load test configuration.

Tip: Use call when you need fresh data for each scenario.
Use callonce when the data can be reused across all scenarios in the feature.

10.

KARATE AUTOMATION NOTES

19. Data-Driven Testing

- One test can run multiple times with different input data.
- Data can come from external JSON or CSV files.
- Helps avoid duplication and makes tests more maintainable.

Example JSON (data/users.json)

```
[
  {
    "id": 1,
    "name": "John",
    "email": "john@test.com"
  },
  {
    "id": 2,
    "name": "Alice",
    "email": "alice@test.com"
  }
]
```

Karate Feature Snippet

```
Feature: Create users from external data

  Scenario: Create multiple users
    * def users = read("data/users.json")
    * def baseUrl = 'https://reqres.in'

    * foreach(user in users)
        Given url baseUrl + '/api/users'
        And request user
        When method post
        Then status 201
        * print 'Created user: ' + user.name
```

Data-Driven Testing Flow

External Data (JSON / CSV) → Feature File (Reads & Loops) → Multiple Test Runs

20. Scenario Outline & Examples

- Use placeholders like <id> and <status> in the Scenario Outline.
- Each row in the Examples table runs the scenario once with different values.

```
Feature: Get user by id

  Scenario Outline: Get user and validate status
    Given url "https://reqres.in/api/users/<id>'
    When method get
    Then status <status>
    And match responseStatus == <status>

  Examples:
```

id	status
1	200
2	200
999	404

Scenario Outline Flow

Scenario Outline (With placeholders <id>, <status>) → Examples (Multiple rows of test data) → Multiple Executions (One per row)

@aman_views

KARATE AUTOMATION NOTES

21. Authentication

- Authentication is used to verify the identity of a user or application before accessing protected resources.
- Common authentication types in APIs:
  - Basic Auth – username and password (encoded in Base64).
  - Bearer / JWT – token based authentication (most common).
  - API Key – key passed in header or query parameter.
  - OAuth 2.0 – industry standard for secure authorization.
- APIs may require different auth mechanisms depending on the application and security requirements.

Example: Bearer Token Authentication

```
Feature: Access protected API with Bearer token

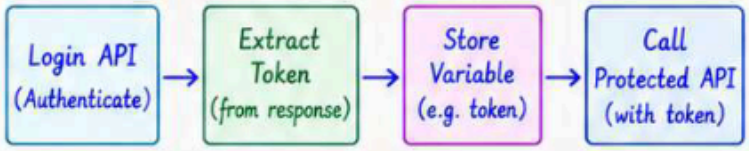
Scenario: Call API with Bearer token
* def token = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'
Given url 'https://api.example.com/users'
And header Authorization = 'Bearer ' + token
When method get
Then status 200
And match response.success == true
```

Note: The token value shown above is just an example. APIs may require different authentication mechanisms such as Basic Auth, API Key or OAuth 2.0.

22. Dynamic Token Handling

- Call login API with required credentials.
- Extract token from the response.
- Store token in a variable.
- Reuse the token in later requests (for protected APIs).

Flow: Dynamic Token Handling



1. Login Request (Get Token)

```
Feature: Login and get token

Scenario: Login to application
Given url "https://api.example.com/auth/login'
And request
  {
    "username": "admin",
    "password": "admin123"
  }
When method post
Then status 200
* def token = response.token
```

2. Reuse Token in Another Request

```
Feature: Use token for protected API

Scenario: Get user details
Given url "https://api.example.com/users/1'
And header Authorization = "Bearer ' + token
When method get
Then status 200
And match response.id == 1
And match response.name == 'John Doe'
```

@aman_views

KARATE AUTOMATION NOTES

23. JSONPath & Response Extraction

- JSON values can be extracted from a response and reused in validations or later requests.
- Extracted values can be stored in variables and used in the next API call.

Sample JSON Response

```
{
   "id": 101,
   "name": "John Doe",
   "address": {
      "city": "New York",
      "zip": "10001"
   },
   "roles": ["admin", "user"],
   "active": true
}
```

Extraction Example

```
* def id = response.id
  # id = 101
* def city = response.address.city
  # city = "New York"
* def firstRole = response.roles[0]
  # firstRole = "admin"
```

Note:
Extracted values can be used in the next API call, validations or in any later step of the scenario.

24. Conditional Logic & JavaScript

- Karate supports JavaScript for dynamic logic and decision making.
- You can also create reusable helper functions using JavaScript.

If / Else Example

```
* def responseStatus = response.status
* if (responseStatus == 200) {
     karate.log('Request successful')
  } else {
     karate.log('Request failed with status: '
                + responseStatus)
  }
```

Useful Built-in Functions

karate.log()	Prints values to console
karate.sizeOf()	Returns size of array or object
karate.typeOf()	Returns the data type

Function Example

```
# function to concatenate names
* def getFullName = function(first, last) {
     return first + ' ' + last
  }
* def name = getFullName('John', 'Doe')
* karate.log(name)  # John Doe

# function to check empty string
* def isEmpty = function(str) {
     return !str || str.trim() == ''
  }
* karate.log(isEmpty(''))  # true
```

KARATE AUTOMATION NOTES

25. Background & Hooks

- Background runs before every scenario in a feature and is good for common setup such as base URL and common headers.
- Hooks / config can be used for after-scenario or after-feature actions.

Example: Background in feature file

```
Feature: User API
  Background:
    * url 'https://regres.in'
    * configure headers = {
        "Content-Type": "application/json",
        "Accept": "application/json"
      }

  Scenario: Get user by id
    Given path 'api', 'users', 2
    When method get
    Then status 200
    And match response.data.id == 2
```

Example: Hooks / Configure in feature file

```
Feature: Hooks Example

  # runs after each scenario
  * configure afterScenario = function(){
      karate.log('After Scenario: ' +
                 karate.scenario.name)
  }

  # runs after entire feature
  * configure afterFeature = function(){
      karate.log('After Feature: ' +
                   karate.feature.name)
  }
```

26. Tags & Test Selection

- Tags like @smoke, @regression, @api, @ignore can be used to categorize scenarios.
- They help in selective test execution.
- You can run only specific tags (e.g. @smoke) and exclude tags (e.g. ~@ignore).

Example: Tagged scenarios

```
@smoke @api
Scenario: Create a user
  Given path 'api', 'users'
  And request { name: 'John', job: 'QA' }
  When method post
  Then status 201

@regression
Scenario: Get all users
  Given path 'api', 'users'
  When method get
  Then status 200
```

Example: JUnit 5 Runner with tag

```
import com.intuit.karate.junit5.Karate;
import org.junit.jupiter-api.Test;

class RunSmokeTest {
    @Test
    Karate testSmoke() {
      return Karate.run("classpath:features")
            .tags("@smoke")
            .relativeTo(getClass());
    }
}
```

Why tags are useful ?

- Run only relevant tests (e.g. smoke tests).
- Separate tests by type (smoke, regression, api).
- Exclude unwanted tests (e.g. @ignore).
- Helpful in CI/CD pipelines for faster execution.

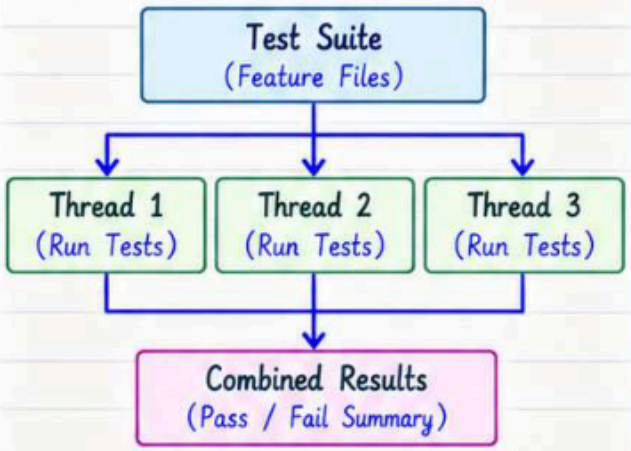
14.

KARATE AUTOMATION NOTES

27. Parallel Execution

- Karate can run tests in parallel to reduce the total execution time.
- Tests should be independent and avoid shared mutable data (e.g. shared files, static variables, common test data).
- Use parallel execution especially for large test suites to improve performance.

Parallel Execution Flow



Runner Example (Parallel Execution)

```
import com.intuit.karate.junit5.Karate;
class ParallelTestRunner {

    @org.junit.jupiter.api.Test
    void runParallelTests() {
        // run all feature files in parallel with 5 threads
        Karate.run("classpath:features").parallel(5);
    }
}
```

Note:

- parallel(5) runs tests using 5 threads.
- If any test fails, the final result will show the fail count and error messages in the report (e.g. 1 failed, 20 passed) with details of the failed scenarios.

28. Reporting & CI/CD

- Karate automatically generates detailed HTML reports after test execution.
- Reports include feature-wise and scenario-wise results, request/response details, logs and screenshots (if configured).
- Karate can be easily integrated with CI/CD tools like Jenkins, GitHub Actions and Azure DevOps.
- The CI pipeline should fail when tests fail, so that issues are caught early.

CI/CD Pipeline Flow



Note:

- Configure the CI job to run: mvn clean test (or appropriate Maven command).
- The build should be marked as FAILED when any test fails.

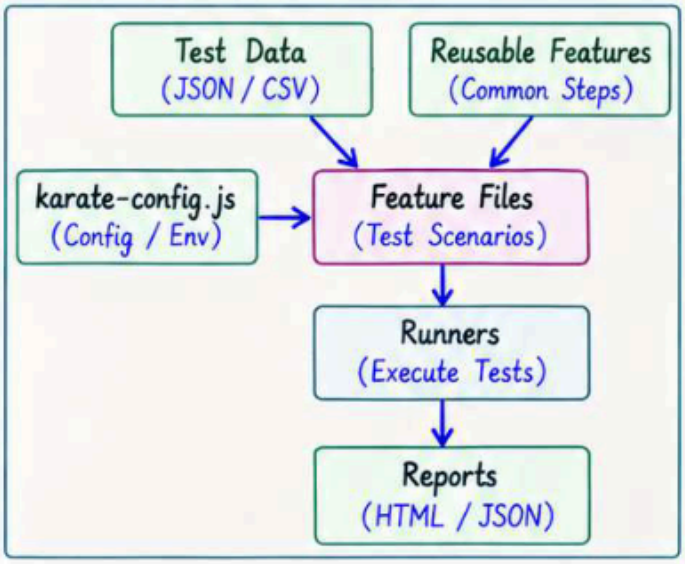
15.

KARATE AUTOMATION NOTES

29. Karate Framework Architecture

- A practical Karate framework separates configuration, features, reusable features, test data, payloads, runners and reports.
- This structure keeps tests maintainable, scalable and easy to reuse across projects.
- Feature files contain actual test scenarios and use test data, payloads and reusable features to perform API/UI tests.
- Runners execute the features (using tags or specific paths) and generate reports.

1. Architecture Flow



2. Sample Folder Structure

```
karate-project/
├─ src
│  └─ test
│     ├─ java
│     │  └─ runners          # Test runners
│     └─ resources
│        ├─ features         # Feature files
│        ├─ common           # Reusable features
│        ├─ data             # Test data (JSON/CSV)
│        ├─ payloads         # Request payloads
│        └─ karate-config.js # Global config
├─ target
│  └─ karate-reports         # HTML reports
└─ pom.xml                   # Maven dependencies
```

30. Best Practices & Interview Revision

1. Best Practices Checklist

- Keep base URLs and environment details in karate-config.js.
- Avoid hardcoded tokens, credentials or sensitive data in feature files.
- Reuse common flows using reusable features (call / callonce).
- Keep scenarios independent and self-contained.
- Use tags (@smoke, @regression, @api etc.) to organize and run tests.
- Keep tests parallel-safe (avoid shared state).

2. Quick Revision Table

Term	Meaning
match	Used for assertions (validate response).
read()	Reads external files (JSON, CSV, text).
call	Calls a reusable feature file (execute steps).
callonce	Calls a reusable feature only once (one-time setup).
karate-config.js	Global configuration (base URL, env, headers, tokens, etc.).

3. Final Takeaway

A well-structured Karate framework with reusable features, clean configuration and good practices leads to maintainable, scalable and reliable automation tests.